NEXA Digital School — Salon Your Future, Lyon

# NEXI : INFECTION
# LE GUIDE DU CODE

Tout le jeu tient dans un seul fichier, `index.html`. Ce guide t'explique à quoi sert chaque partie, comment lire une ligne de code, et où aller pour changer ce que tu veux.

> COMMENT T'EN SERVIR
> Les boutons `Ouvrir dans l'atelier →` ouvrent l'atelier directement sur la bonne ligne, avec la valeur déjà sélectionnée : tu n'as plus qu'à taper. Dans le PDF, ils fonctionnent aussi (il faut Internet).

## Sommaire


1. L'atelier en 2 minutes
2. Les bases du code
3. Les missions de l'atelier
4. La carte du fichier
5. Les réglages ★ Débutant
6. Les données ★★ Codeur
7. Dessins, couleurs et animations
8. Le moteur du jeu
9. Je veux… : les recettes (lave, monstre, boss pas à pas)
10. Ça ne marche plus ?
11. Petit glossaire

# 1. L'atelier en 2 minutes

L'atelier est une page web avec le code du jeu à gauche et le jeu à droite.

| Bouton | Ce qu'il fait |
|---|---|
| ▶ **LANCER** (ou Ctrl + Entrée) | Relance le jeu avec ton code. Si tu as fait une faute, un message rouge indique la ligne. |
| **Aller à...** | Saute à une partie du code (chaque partie commence par un titre ★ ou ★★). |
| **Liste des réglages** (sous le jeu) | Chaque réglage avec sa valeur actuelle et un lien « ligne N » : un clic sélectionne la valeur à changer. |
| **JOUER SUR MON TÉLÉPHONE** | Affiche un QR code : scanne-le, tu joues à *ta* version et tu peux la garder (le lien contient tes modifications). |
| **TÉLÉCHARGER MON CODE** | Enregistre ton fichier `nexi_infection_ma_version.html` : double-clic dessus pour jouer, ouvre-le dans un éditeur de texte pour continuer à coder chez toi. |
| **NOUVEAU VISITEUR** | Efface toutes les modifications et revient au code d'origine. |

> ASTUCE
> Clique une fois dans le jeu avant de jouer au clavier : flèches ou ZQSD pour bouger, Espace pour sauter, J pour attaquer, K pour la compétence, Échap ou P pour la pause.

## 2. Les bases du code

Le jeu est écrit en **JavaScript**, le langage des pages web. Pas besoin de tout comprendre pour le modifier : voici les 8 choses que tu vas croiser partout, avec de vrais exemples tirés du jeu.

### Le commentaire : du texte pour les humains

```
// Points de vie. Essaie 999 pour un Nexi quasi invincible.
```

Tout ce qui suit `//` sur une ligne est ignoré par l'ordinateur. Ce sont des notes pour toi. Les gros blocs entre `/*` et `*/` aussi.

### La constante : un nom qui retient une valeur

```
const MESSAGE_KO = "SYSTEM FAILURE";
```

`const` crée une boîte nommée `MESSAGE_KO` qui contient le texte `"SYSTEM FAILURE"`. Partout où le jeu écrit `MESSAGE_KO`, il utilise ce texte. Le `;` termine l'instruction.

### Les 3 types de valeurs que tu vas changer

| Type | Exemple | Règle |
|---|---|---|
| **Nombre** | `100`, `0.22`, `-5` | Sans guillemets. La virgule décimale s'écrit avec un **point** : `0.22`, pas `0,22`. |
| **Texte** (chaîne) | `"Cheval de Troie"` | Toujours entre guillemets `" "`, au début *et* à la fin. |
| **Booléen** | `true` / `false` | Vrai ou faux, sans guillemets. Sert d'interrupteur. |

### L'objet : une fiche avec des cases nommées

```
const JOUEUR = {
  PV_MAX: 100,          // Points de vie
  VITESSE: 72,          // Vitesse de déplacement
  ONE_SHOT: false,      // true = chaque coup tue l'ennemi
};
```

Entre `{` et `}`, chaque ligne est un couple `nom: valeur,`. Le jeu lit la valeur avec un point : `JOUEUR.PV_MAX` vaut `100`. **La virgule à la fin de chaque ligne est obligatoire** (sauf parfois la dernière).

### Le tableau : une liste rangée

```
PETITS: ["ver", "glitch", "spyware", "script", "drone"],
```

Entre `[` et `]`, des valeurs séparées par des virgules. Ici, la liste des petits ennemis qui peuvent apparaître. Retire `"drone"` et il n'y aura plus de drones.

### La fonction : une recette que le jeu peut suivre

```
function px(x, y, c) { g.fillStyle = c; g.fillRect(x | 0, y | 0, 1, 1); }
```

`px` dessine **un pixel** de couleur `c` à la position `x`, `y` (`x | 0` arrondit à un nombre entier). Le jeu appelle `px(10, 20, "#ffffff")` pour poser un pixel blanc. Tout le jeu est fait de fonctions comme celle-ci, qui s'appellent entre elles.

### La condition : « si… alors… »

```
if (J.pv <= 0) mourir();
```

« Si les PV de Nexi (`J.pv`) sont inférieurs ou égaux à 0, alors appeler la fonction `mourir` ». Les comparaisons : `===` égal, `!==` différent, `<` plus petit, `>` plus grand.

### Les couleurs : le code hexadécimal

```
P:"#7b3fe0"
```

Une couleur s'écrit `#` suivi de 6 caractères : 2 pour le **rouge**, 2 pour le **vert**, 2 pour le **bleu**, de `00` (rien) à `ff` (maximum). `#ff0000` = rouge pur, `#000000` = noir, `#ffffff` = blanc. Ici `7b` de rouge, `3f` de vert, `e0` de bleu : le violet de Nexi.

## 3. Les missions de l'atelier

Cinq défis pour découvrir le code pas à pas. Chacun prend 2 à 5 minutes. Après chaque modification : ▶ **LANCER**.

### Mission 1 — Nexi surpuissant (un nombre)

Change les points de vie de 100 à 999, puis la hauteur du saut de 215 à 350.

```
PV_MAX: 999,
FORCE_SAUT: 350,
```

**Ce que tu dois voir :** la barre de vie affiche 999, et Nexi saute très haut.
**Tu as appris :** un nombre dans un objet, et la virgule qui termine la ligne.

Ouvrir PV_MAX dans l'atelier → | Ouvrir FORCE_SAUT →

### Mission 2 — Renomme un boss (un texte)

```
troie: "Bourrin 3000",
```

**Ce que tu dois voir :** le nom en grand quand le premier mini-boss arrive (salle 7). Tu peux aussi changer le texte de la mort : `MESSAGE_KO = "GAME OVER";`
**Tu as appris :** un texte s'écrit toujours entre guillemets.

Ouvrir le nom du boss → | Ouvrir le texte de mort →

## Mission 3 — Change les règles (un booléen, un choix)

```
ONE_SHOT: true,
const ELEMENT = "feu";
```

**Ce que tu dois voir :** avec `ONE_SHOT: true`, chaque ennemi touché meurt d'un coup. Avec `"feu"`, tes coups brûlent (les choix possibles sont écrits juste au-dessus : `"aucun"`, `"feu"`, `"poison"`, `"foudre"`, `"glace"`, `"vampire"`).
**Tu as appris :** `true` / `false` sont des interrupteurs ; un texte doit parfois être pris dans une liste précise.

Ouvrir ONE_SHOT → | Ouvrir ELEMENT →

## Mission 4 — Repeins Nexi (une couleur)

```
const PAL_NEXI = { K:"#14082c", P:"#e03f3f", L:"#ff7b7b", d:"#a81f1f", …
```

`P` est le violet principal du corps, `L` sa version claire, `d` sa version sombre.
Change les trois avec la même teinte, plus claire et plus sombre : Nexi devient rouge (ou la couleur de ton choix).
**Tu as appris :** les couleurs hexadécimales, et qu'une lettre du dessin correspond à une couleur de la palette.

Ouvrir la couleur de Nexi →

## Mission 5 (bonus) — Crée ton propre bonus de porte (une liste d'objets)

Dans `LISTE_BONUS`, copie une ligne, colle-la juste en dessous et change-la :

```
{ nom: "Mode turbo", desc: "Vitesse +80 %", duree: 3, mods: { vitesse: 1.8 } },
```

**Ce que tu dois voir :** en passant les portes, ton bonus finit par tomber (bandeau « BONUS : Mode turbo »). Pour le tester tout de suite, mets `CHANCE_BONUS: 1` dans `PORTES`.
**Tu as appris :** une liste d'objets, et qu'on code souvent en copiant puis en adaptant une ligne qui marche.

Ouvrir LISTE_BONUS →

## Mission 6 (bonus) — Invente ton propre élément

Dans `EFFETS_ELEMENTS`, copie la ligne `feu`, colle-la en dessous et transforme-la :

```
  plasma: { effet: "brulure", couleur: "#ff3300", magique: true, degatsParSec: 20, duree: 5 },
```

Puis, tout en haut du fichier : `const ELEMENT = "plasma";`

**Ce que tu dois voir :** le nom « plasma » en rouge sous la barre de vie, et des ennemis qui fondent. Si tu lis « ELEMENT INCONNU », les deux noms ne sont pas écrits pareil.
**Tu as appris :** le nom d'une ligne sert d'identifiant ailleurs dans le code ; il faut l'écrire exactement pareil aux deux endroits.

Ouvrir EFFETS_ELEMENTS →

## 4. La carte du fichier

Le fichier fait environ 4 500 lignes, rangées du plus simple au plus technique. Les titres en `★ DÉBUTANT` et `★★ CODEUR` servent de panneaux indicateurs (menu « Aller à… » de l'atelier).

### ★ DÉBUTANT (début du fichier)

Les réglages simples : vie, vitesse, compétence, monstres, noms des boss, histoire, progression. Que des nombres, des textes et des true/false.

### ★★ CODEUR — LES DONNÉES

Les listes du jeu : portes, butin, bestiaire, salles, boss, virus, armes, armures, poids, musique, bruitages, pièges, monde.

### ★★ CODEUR — LES DESSINS

Le pantin de Nexi (poses), les palettes de couleurs, les sprites en lettres, les boss dessinés avec des formes.

### ★★ CODEUR — LE MOTEUR

Les fonctions qui font vivre le jeu : salles, ennemis, combat, équipement, sauvegarde, son, boss, mise en scène.

### ★★ CODEUR — AFFICHAGE ET DÉMARRAGE

Le dessin de chaque image, les boutons tactiles, le clavier, la boucle de jeu et le lancement.

> **LA RÈGLE D'OR**
> Plus tu descends dans le fichier, plus le code est technique. Les 150 premières lignes suffisent pour transformer le jeu.

# 5. Les réglages ★ Débutant

## CONFIGURATION DU JOUEUR (Nexi)

| Réglage | Départ | Ce qu'il change | Essaie |
|---|---|---|---|
| `PV_MAX` | 100 | Les points de vie maximum. | 999 |
| `VITESSE` | 72 | La vitesse de déplacement, en pixels par seconde. | 140 ou 35 |
| `FORCE_SAUT` | 215 | La force du saut (vers le haut). | 300 |
| `DEGATS_LEGER` | 10 | Les dégâts des coups normaux (toutes les armes en tiennent compte). | 30 |
| `DEGATS_COMBO` | 18 | Les dégâts du 3e coup du combo à l'épée. | 50 |
| `COOLDOWN_LEGER` | 0.22 | Le temps (en secondes) entre deux coups. Plus petit = plus de coups par seconde. | 0.05 |
| `REGEN_PV` | 0 | PV rendus à chaque pulsation. | 5 |
| `PULSATION_PV` | 6 | Toutes les combien de secondes les soins et pertes lents s'appliquent. | 2 |
| `ONE_SHOT` | false | true = chaque coup qui touche tue l'ennemi. | true |
| `UN_COUP_KO` | false | true = mode hardcore : Nexi meurt au premier coup. | true |

Ouvrir dans l'atelier →

## LA COMPÉTENCE : ONDE N'SHIELD

La touche K (bouton vert) libère une onde autour de Nexi. Elle touche les Spectres et les rend tangibles 3 secondes.

| | |
|---|---|
| `DEGATS: 14` | Dégâts de l'onde. |
| `RAYON: 40` | Taille de l'onde en pixels (essaie 120). |
| `ETOURDISSEMENT: 1` | Secondes pendant lesquelles les ennemis touchés sont immobiles. |
| `RECUL: 14` | Distance à laquelle ils sont repoussés. |
| `COOLDOWN: 4` | Secondes avant de pouvoir la relancer (essaie 0.5). |

`Ouvrir dans l'atelier →`

### ÉLÉMENT DES ATTAQUES

```
const ELEMENT = "aucun";
```

Remplace le mot entre guillemets par `"feu"` (brûlure), `"poison"` (se cumule à chaque coup), `"foudre"` (paralyse au bout de 3 coups), `"glace"` (ralentit) ou `"vampire"` (Nexi récupère de la vie en frappant). Garde les guillemets et le `;`. Si le nom n'existe pas, le jeu affiche « ELEMENT INCONNU » sous la barre de vie et joue sans élément. Tu peux aussi inventer le tien : voir Les éléments.

### DIFFICULTÉ DES MONSTRES

| | |
|---|---|
| `MULT_PV: 1` | Multiplie les PV de tous les ennemis (3 = trois fois plus résistants). |
| `MULT_DEGATS: 1` | Multiplie leurs dégâts (0.5 = partie tranquille). |
| `GAIN_PAR_SALLE: 0.045` | À chaque salle, les ennemis deviennent 4,5 % plus forts. Ça se cumule : en salle 30, ils sont 3,6 fois plus forts. |
| `ENNEMIS_DEPART: 2` / `ENNEMIS_MAX: 9` | Le nombre d'ennemis dans la 1re salle, et le maximum. |
| `CHANCE_ELEMENTAIRE: 0.15` | 15 % des ennemis sont de feu ou de foudre (halo coloré), à partir de la salle 3. |

### NOMS DES BOSS ET HISTOIRE

`NOMS_BOSS` et `TITRES_BOSS` : les noms affichés en grand à l'arrivée de chaque boss. `HISTOIRE.RENCONTRES_AVANT_VER_ULTIME` : combien de fois on

affronte le Tyran avant de trouver l'Abyssal Worm (mets 1 pour voir la fin plus vite).

## PROGRESSION DE NEXI

| | |
|---|---|
| `XP_PREMIER_NIVEAU: 20` | XP pour passer niveau 2 (chaque niveau suivant demande `XP_EN_PLUS_PAR_NIVEAU` de plus). |
| `FRAGMENTS_GARDES_A_LA_MORT: 0.25` | Part des fragments conservée quand Nexi meurt. |
| `TAILLE_SAC: 6` / `TAILLE_COFFRE: 12` | Places dans le sac et dans le coffre du Terminal. |
| `SOIN_FRAGMENTS_PAR_PV: 0.4` | Prix du soin à la porte, par PV manquant. |
| `COUT_AMELIORATION: 25` / `HAUSSE_COUT: 1.6` | Prix d'une amélioration au Terminal : 25, puis ×1,6 à chaque fois (25, 40, 64, 102…). |
| `CHARGE_DE_BASE: 30` | Le poids que Nexi porte sans être ralenti au départ. |
| `SAUVEGARDE_AUTO: true` | false = rien n'est gardé dans le navigateur. |

`GAIN_PAR_POINT` dit ce que rapporte 1 point dépensé au Terminal : +10 PV, +8 % de dégâts, +4 % de vitesse, +4 de défense, ou +3 de charge (endurance).

# 6. Les données ★★ Codeur

Ces sections décrivent le contenu du jeu sous forme de listes. Pour chacune : à quoi elle sert, comment lire une ligne, ce que tu peux changer sans risque.

### TEXTES DE MORT

`MESSAGE_KO` et `SOUS_TITRE_KO` : le texte de l'écran de mort.

### LES ÉLÉMENTS

```
glace: { effet: "ralentir", couleur: "#9fe8ff", magique: true, force: 0.5, duree: 3
},
```

Une ligne = un élément. Le nom avant les deux-points ( `glace` ) est celui à écrire dans `const ELEMENT = "glace";` . Chaque élément choisit un `effet` parmi les 6 que le moteur sait faire :

| effet | Ce qu'il fait | Ses réglages |
|---|---|---|
| `"brulure"` | Dégâts continus pendant quelques secondes. | `degatsParSec` , `duree` |
| `"cumul"` | Chaque coup ajoute une dose ; plus de doses = plus de dégâts. | `degatsParSec` (par dose), `duree` , `cumulMax` |
| `"paralysie"` | Remplit une jauge ; pleine, l'ennemi est figé. | `coupsPourParalyser` , `dureeParalysie` , `degatsBonus` |
| `"ralentir"` | L'ennemi bouge et attaque moins vite (deux fois moins fort sur les boss). | `force` (0.5 = moitié moins vite, 0.9 au maximum), `duree` |
| `"volDeVie"` | Nexi récupère une part des dégâts qu'il inflige. | `part` (0.15 = 15 %) |
| `"flaque"` | Chaque coup laisse une flaque au sol (la lave) : les ennemis dedans brûlent et ralentissent. | `degatsParSec` , `force` , `duree` (vie de la flaque), `largeur` |

Deux options de plus : `magique: true` permet de blesser les Spectres sans compétence, `spectresImmunises: true` rend les Spectres insensibles à l'élément. `couleur` colore le nom de l'élément, les particules et l'onde.

> ### À TOI
>
> Pour créer ton élément : copie une ligne, change le nom, l'effet, les chiffres et la couleur, puis écris ce nom dans `ELEMENT` (mission 6). Ajouter un 7e type d'effet demande d'écrire du code dans la fonction `appliquerElementType` : c'est un vrai défi de codeur. La recette A (section 9) montre comment la flaque de lave a été ajoutée.

## LES PORTES (bonus et malus)

```
{ nom: "Surcharge offensive", desc: "Dégâts +20 %", duree: 3, mods: { degats: 1.2 }
},
```

À chaque porte, le jeu tire un bonus (liste `LISTE_BONUS`) ou un malus (`LISTE_MALUS`). `duree` = nombre de salles. `mods` multiplie une caractéristique (`vitesse`, `degats`, `cadence`, `cooldownLourd`, `pvEnnemis`). `ajouts` ajoute une valeur (`pvParPulsation`). `drapeaux` active un effet spécial déjà codé (`gardien`, `foudreAuto`, `parasite`…). `PORTES.CHANCE_BONUS` règle la chance d'avoir un bonus. Dans la salle de repos, bonus et malus sont en pause.

## LOOT (le butin)

Les chances (entre 0 et 1) : fiole de soin, coffre dans une salle, objet lâché par un ennemi, arme enchantée de feu ou de foudre. `NIVEAU_TOUTES_LES: 12` : les objets trouvés gagnent 1 niveau toutes les 12 salles.

## BESTIAIRE

```
ver: { nom: "Ver informatique", pv: 18, degats: 10, vitesse: 30, apparitionDes: 1,
poids: 3 },
```

Une ligne par ennemi. `apparitionDes` = première salle où il peut apparaître, `poids` = sa fréquence (plus grand = plus courant). `spectre: true` le rend résistant aux coups physiques, `defense: 0.5` lui fait perdre moitié moins de PV. Le comportement de chaque ennemi est codé dans la fonction `majEnnemi`. Pour inventer un ennemi : `comme` (copier un ennemi existant), `teinte` (changer ses couleurs), `dessin` et `palette` (son propre dessin). Voir la recette B, section 9.

## PROGRESSION DES SALLES

`PROGRESSION.CYCLE` décrit un cycle de 6 salles : on ajoute des petits ennemis, puis des gros, puis le mini-boss (`elite: true`). Juste avant chaque mini-boss, le

jeu insère une salle de repos (Terminal). Chaque nouveau cycle recommence avec un ennemi de plus.

## LES BOSS et LES VIRUS

`BOSS` donne les PV et les dégâts de base de chaque boss. `GAIN_PAR_CYCLE: 0.55` : à chaque cycle, leurs PV sont multipliés par 1,55 (et leurs dégâts par 1,25). `ARCHONTE_TOUS_LES: 2` : le Tyran revient tous les 2 cycles. Chaque mini-boss vaincu lâche un **virus** (liste `VIRUS`) : une compétence qui remplace l'onde, avec son avantage (`effet`), son défaut (`contre`) et son temps de recharge. Pour inventer un mini-boss à partir d'un boss existant : `comme`, `teinte` et `titre`, puis son nom dans `ORDRE` (recette C, section 9).

## LES ARMES

```
katana: { nom: "Katana Fibre", cat: "epee", poids: 3, degats: 10, gain: 1.5,
portee: 1.4, desc: "Lame fine, portée plus longue" },
```

| | |
|---|---|
| `cat` | La catégorie, qui décide de la façon de frapper : `"epee"` (combo de 3 coups), `"fouet"` (2 coups longs), `"lourde"` (1 grand coup lent), `"distance"` (tirs, avec `forme` `"arc"` ou `"baton"`). |
| `degats` / `gain` | Dégâts au niveau 1, puis dégâts en plus à chaque niveau. |
| `poids` | Compte dans la charge de Nexi. |
| `vitesse` / `portee` | Multiplicateurs (1 = normal). |
| `element` / `effet` | Feu ou foudre intégrés ; effet spécial déjà codé (`"parefeu"`, `"chaine"`, `"onde"`, `"minage"`…). |
| `corrompue: true` | Fait partie du set corrompu lâché par le Tyran. |

## LES ARMURES et LE POIDS

Chaque armure a une `piece` (casque, plastron, gantelets, bottes), une `defense` et un `poids`. Les dégâts reçus sont multipliés par `50 / (50 + défense)` : avec 50 de défense, Nexi prend moitié moins. La section `CHARGE` règle les seuils : jusqu'à 30 % de la charge max, Nexi est léger (+10 % de vitesse) ; au-delà de 70 %, il est lourd (−20 % de vitesse et d'attaque). Le set corrompu complet (4 armures + 1 arme corrompue) active une **synergie** selon le virus équipé (liste `SYNERGIES`).

## LA MUSIQUE et LES BRUITAGES

```
exploration: { tempo: 116, ondeMelodie: "square",
  melodie: "E4 - - G4 - - B4 - A4 - G4 - F#4 - - - …",
```

Aucun fichier son : le navigateur fabrique les notes. Une note = une croche : `E4` = mi de l'octave 4, `F#4` = fa dièse, `-` tient la note, `.` = silence. Batterie : `k` grosse caisse, `s` caisse claire, `h` charleston. `tempo` = vitesse en battements par minute. Dans `BRUITAGES`, `de` et `a` sont les fréquences de départ et d'arrivée d'un son (en hertz).

## LES PIÈGES et LE MONDE

`PIEGES` règle la chance d'avoir des pièges dans une salle et la force de chaque type : la mine ( `explosif` ), la grille de flammes ( `brulant` ) et l'arc électrique ( `foudroyant` ), avec leurs temps `actif` et `repos`. `MONDE.GRAVITE: 540` : la force qui ramène Nexi au sol (essaie 200 pour la Lune). `LARGEUR_SALLE: 560` : la longueur des salles en pixels.

# 7. Dessins, couleurs et animations

## Le pantin de Nexi : les poses

```
leger2: { brasB: 160, brasF: 5, jambeB: 125, jambeF: 50, epee: 0, dx: 1, yeux:
"plisses" },
```

Nexi est un pantin : chaque pose donne l'angle (en degrés) de ses bras, jambes et de son arme. 0 = vers la droite, 90 = vers le bas, -90 = vers le haut. `dx` / `dy` décalent le corps, `yeux: "plisses"` plisse les yeux. Change un angle de la pose `repos1` et Nexi se tient autrement. Le dessin à partir de ces angles est fait par la fonction `rendrePose`.

## Les palettes et les sprites en lettres

```
const PAL_ZOMBIE = { L:"#e4dcff", l:"#a796e0", d:"#6a58a8", D:"#0c0716",
E:"#FF5E7A", … };
const ZOMBIE = ["........LLLL........", ".....LLllllLL.....", …
```

Les petits ennemis sont dessinés en **texte** : chaque chaîne est une ligne de pixels, chaque lettre une couleur de la palette, le point `.` un pixel transparent. Change `E:"#FF5E7A"` (les yeux du zombie) en `"#3DF5C4"` et ses yeux deviennent verts. Un contour sombre est ajouté automatiquement.

## Les boss : des formes géométriques

Les boss sont trop grands pour être dessinés lettre par lettre : ils sont construits avec des formes ( `rect` , `ellipse` , `capsule` , `poly` ) et des matériaux à 3 tons. La fonction `rendreFormes` transforme ces formes en pixels, avec ombres et contour automatiques. Les tailles de chaque boss sont dans `TAILLES_BOSS` .

# 8. Le moteur du jeu

Pour ceux qui veulent comprendre comment le jeu fonctionne vraiment.

### La boucle de jeu : 60 fois par seconde

```
boucle()  ── appelée environ 60 fois par seconde par le navigateur
  └─ image()
       ├─ lireTouches()   quelles touches / quels boutons sont appuyés ?
       ├─ maj(dt)         faire avancer le monde de dt secondes (≈ 0,016)
       │    ├─ majJoueur     déplacements, sauts, attaques de Nexi
       │    ├─ majEnnemi     chaque ennemi bouge et attaque  (majBoss pour
les boss)
       │    ├─ tirs, pièges, butin, Terminal, porte…
       └─ dessiner()      redessiner toute l'image : décor, ennemis, Nexi,
HUD
```

`dt` (« delta temps ») est le temps écoulé depuis l'image précédente. Tout ce qui bouge est multiplié par `dt` : `e.x += e.dir * 130 * dt` veut dire « avancer de 130 pixels par seconde », quelle que soit la vitesse de l'ordinateur.

### Les états du jeu

La variable `etat` dit où on en est : `"titre"`, `"histoire"`, `"jeu"`, `"menu"` (pause, Terminal, inventaire…), `"ko"`, `"victoire"`. Le monde n'avance que quand `etat === "jeu"` : c'est ce qui met le jeu en pause dans les menus.

### Les salles

`infoSalle(n)` calcule ce qu'est la salle numéro `n` (normale, Terminal, mini-boss, Tyran, sanctuaire) en rejouant les cycles depuis le début. `chargerSalle(n)` vide la salle précédente et crée la nouvelle : ennemis ( `compositionSalle` ), coffre, pièges, boss. `salleSuivante()` est appelée quand Nexi passe la porte : elle tire un bonus ou un malus puis charge la salle.

### Le combat

```
function infligerDegats(e, dmg, fort, discret, source) {
  …
  dmg *= 1 - e.defense;
  if (e.vulnerable > 0) dmg *= 2;
  …
  e.pv -= dmg;
```

Quand l'arme de Nexi touche un ennemi, `toucherAvecArme` vérifie les cas spéciaux (Spectres, boss 404, effets d'arme) puis appelle `infligerDegats`, qui retire les PV en tenant compte de la défense et des faiblesses. Les dégâts de l'arme viennent de `degatsArme` : niveau de l'arme × réglages Débutant × points de niveau × bonus. Dans l'autre sens, `blesserJoueur` blesse Nexi, en tenant compte de son armure.

## Les boss : des attaques au choix

```
troie: ["ruee", "martelage", "ventre"],
```

`KITS` donne la liste des attaques de chaque boss ; le boss en pioche une, l'exécute, fait une pause, puis recommence. Chaque attaque est une fonction de `ATTAQUES`, appelée à chaque image tant qu'elle n'est pas finie (elle renvoie `true` quand elle a terminé). Par exemple, la ruée du Cheval de Troie : il gratte le sol 0,9 seconde (un « ! » apparaît), puis fonce à 130 pixels par seconde jusqu'au mur.

## Équipement, Terminal et menus

Un objet est une petite fiche `{ id, type, niv }` (et parfois `element`). `equiper` le place sur Nexi et range l'ancien dans le sac. Tous les menus (pause, Terminal, inventaire, réglages) passent par `ouvrirMenu`, qui reçoit un titre et une liste de lignes : chaque ligne a un texte et une `action` (la fonction lancée quand on la choisit).

## La sauvegarde

Au Terminal, `instantane()` prend une « photo » de la partie, `encoderSauvegarde()` la transforme en code `NEXI-XXXX-…`, rangé dans le navigateur ( `localStorage` ). En pleine descente, la partie n'est pas sauvegardée : quitter compte comme une mort. Le même code peut s'afficher en QR code, se copier ou se télécharger.

## Le son et l'affichage

La musique est jouée par `planifierMusique`, qui prépare les notes un peu en avance pour un rythme parfait ; `son("coup")` joue un bruitage. Côté image, tout est dessiné en basse résolution (environ 280 × 130 pixels) puis agrandi sans flou : c'est ce qui donne le style pixel art. `rect`, `px` et `texte` sont les trois fonctions de dessin de base.

## 9. Je veux… : les recettes

| Je veux… | Je vais où | Je fais quoi |
|---|---|---|
| un Nexi plus fort | `JOUEUR` | Monte `PV_MAX`, `DEGATS_LEGER`, baisse `COOLDOWN_LEGER`. |
| changer la couleur de Nexi | `PAL_NEXI` | Change `P`, `L` et `d` (couleur, version claire, version sombre). |
| changer la pose de Nexi au repos | `POSES`, ligne `repos1` | Change les angles `brasF` ou `epee` (de -180 à 180). |
| renommer un boss | `NOMS_BOSS` | Change le texte entre guillemets. |
| un boss plus dur | `BOSS` | Monte son `pv` ou ses `degats` ; `GAIN_PAR_CYCLE` pour tous. |
| voir le Tyran tout de suite | `BOSS` | `ARCHONTE_TOUS_LES: 1` : le Tyran arrive en salle 7. |
| voir la fin du jeu | `HISTOIRE` et `BOSS` | `RENCONTRES_AVANT_VER_ULTIME: 1` et `ARCHONTE_TOUS_LES: 1`. |
| créer une arme | `ARMES` | Copie la ligne `katana`, change le nom de la ligne (`monArme:`), le `nom` et les chiffres. Elle apparaîtra dans les coffres. |
| inventer un élément | `EFFETS_ELEMENTS` | Copie une ligne, renomme-la, choisis un `effet` et une `couleur`, puis écris son nom dans `ELEMENT`. |
| de la lave au sol | `ELEMENT` | `const ELEMENT = "lave";` (recette A). |
| un nouveau monstre | `BESTIAIRE` et `PETITS` | Copie une ligne avec `comme` et `teinte` (recette B). |
| un nouveau mini-boss | `BOSS` et `ORDRE` | Une ligne avec `comme: "troie"` (recette C). |
| plus de coffres | `LOOT` | `CHANCE_COFFRE: 1` = un coffre par salle. |
| moins de pièges | `PIEGES` | `CHANCE_PAR_SALLE: 0` pour les supprimer. |
| un nouveau bonus de porte | `LISTE_BONUS` | Copie une ligne et adapte-la (mission 5). |

| | | |
|---|---|---|
| que des bonus aux portes | `PORTES` | `CHANCE_BONUS: 1`. |
| composer ma musique | `MUSIQUE` | Change `tempo` et les notes d'un thème (garde les espaces entre les notes). |
| changer le texte de mort | `MESSAGE_KO` | Remplace `"SYSTEM FAILURE"`. |
| sauter comme sur la Lune | `MONDE` | `GRAVITE: 200`. |
| des ennemis d'un seul type | `PROGRESSION` | Laisse un seul nom dans `PETITS`, par exemple `["ver"]`. |

## Recettes pas à pas

Le tableau dit où aller. Ces recettes montrent le code complet, prêt à copier : on part toujours d'une ligne qui marche, on la copie, puis on la change.

## Recette A — De la lave qui reste au sol

**Ce que tu vas obtenir :** chaque coup de Nexi laisse une flaque de lave par terre. Les ennemis qui marchent dedans brûlent et sont ralentis. Les ennemis volants passent au-dessus.

La lave existe déjà dans `EFFETS_ELEMENTS` :

```
lave: { effet: "flaque", couleur: "#ff5a1f", magique: true, degatsParSec: 8,
force: 0.4, duree: 4, largeur: 30 },
```

Pour l'activer, tout en haut du fichier : `const ELEMENT = "lave";`

| Réglage | Ce qu'il change |
|---|---|
| `degatsParSec` | Les dégâts par seconde pour un ennemi debout dans la flaque. |
| `force` | Le ralentissement : 0.4 = 40 % moins vite, 0.9 au maximum, 0 = pas de ralentissement. |
| `duree` | Le nombre de secondes avant que la flaque refroidisse. |
| `largeur` | La taille de la flaque, en pixels (Nexi en fait environ 16). |

**Va plus loin :** invente ta propre flaque en copiant la ligne. Une flaque d'acide verte, large et lente :

```
acide: { effet: "flaque", couleur: "#b6ff3a", degatsParSec: 4, force: 0.7,
duree: 6, largeur: 50 },
```

**Une arme de lave :** dans `ARMES`, copie la ligne `overheat` et donne-lui l'élément `"lave"`. Elle apparaîtra dans les coffres, même si `ELEMENT` vaut `"aucun"` :

```
magmaBlade: { nom: "Lame de magma", cat: "epee", poids: 4, degats: 9, gain:
1.3, element: "lave", desc: "Laisse de la lave au sol" },
```

**Comment c'est codé (pour les codeurs) :** trois petites fonctions. `poserFlaque` ajoute une flaque dans la liste `flaques` quand un coup touche (ou ravive celle qui est déjà là). `majFlaques` est appelée 60 fois par seconde : elle vieillit chaque flaque et, pour chaque ennemi qui a les pieds dedans, réutilise la brûlure et le

ralentissement qui existaient déjà. `dessinerFlaques` la dessine avec des bulles.

C'est la bonne façon d'ajouter un effet : réutiliser ce que le moteur sait déjà faire.

Ouvrir EFFETS_ELEMENTS →

## Recette B — Un nouveau monstre

Trois niveaux : fais le premier, puis continue si tu as le temps.

**Niveau 1 : un cousin d'un monstre existant (2 minutes).** Dans `BESTIAIRE`, copie la ligne `ver`, colle-la en dessous et ajoute `comme` et `teinte` :

```
verLave: { nom: "Ver de lave", pv: 25, degats: 12, vitesse: 40, apparitionDes:
1, poids: 3, comme: "ver", teinte: 40 },
```

`comme: "ver"` : il bouge, se dessine et a la taille du ver. `teinte: 40` décale ses couleurs (un nombre de 0 à 360, comme un tour de la roue des couleurs : 180 = couleurs opposées). Ensuite, dans `PROGRESSION`, ajoute son nom dans la liste des petits ennemis, sinon il n'apparaîtra jamais :

```
PETITS: ["verLave", "ver", "glitch", "spyware", "script", "drone"],
```

**Ce que tu dois voir :** des vers orange dès la salle 1. Pour un gros ennemi, mets `comme: "trojan"` et ajoute son nom dans `GROS`.

**Niveau 2 : son propre dessin.** Ajoute une `palette` (une lettre = une couleur) et un `dessin` (une chaîne = une ligne de pixels, `.` = transparent). `dessin2` est la 2e image, pour qu'il s'anime. Sa taille se calcule toute seule d'après le dessin :

```
cookie: { nom: "Cookie traqueur", pv: 20, degats: 8, vitesse: 35,
apparitionDes: 1, poids: 3, comme: "ver",
  palette: { K: "#1a0d05", c: "#d99a4e", p: "#3b1d0a", E: "#FF5E7A" },
  dessin:  ["...KKKKK...", "..KccccK..", ".KccpccpcK.", "KcccEccEccK",
            "KcpccccccpK", "KccccpcccK", ".KccpccccK.", "..KKKKKKK.."],
  dessin2: ["...KKKKK...", "..KccccK..", ".KcpccpccK.", "KcccEccEccK",
            "KccccpcccK.", "KcpccccpcK", ".KcccccpcK.", "..KKKKKKK.."] },
```

N'oublie pas `"cookie"` dans `PETITS`. Astuce : dessine d'abord sur du papier quadrillé, une case = une lettre.

**Niveau 3 : son propre comportement (codeur).** Enlève `comme: "ver"` de la ligne du cookie. Puis, dans la fonction `majEnnemi`, ajoute un `case` à côté de celui du ver. Le cookie saute vers Nexi toutes les 1,5 seconde :

```
    case "cookie": { // saute vers Nexi
      e.dir = Math.sign(dx) || e.dir;              // se tourne vers Nexi
      e.chrono -= dt;                              // compte à rebours avant
le prochain saut
      if (e.chrono <= 0 && e.y >= e.baseY) { e.vy = -170; e.chrono = 1.5; }
      e.vy += MONDE.GRAVITE * dt;                  // la gravité le ramène au
sol
      e.y = Math.min(e.baseY, e.y + e.vy * dt);
      if (e.y < e.baseY) e.x += e.dir * e.vitesse * 2 * dt;   // avance
seulement en l'air
      break;
    }
```

`dx` est la distance jusqu'à Nexi (négative s'il est à gauche), `dt` le temps écoulé depuis la dernière image, `e.baseY` la hauteur du sol pour ce monstre. Change `-170` (la force du saut) ou `1.5` (le temps entre deux sauts) pour voir.

Ouvrir BESTIAIRE →

### Recette C — Un nouveau mini-boss

Un boss a beaucoup de pièces (dessin en formes, attaques, virus lâché à sa mort). Le plus simple : en prendre un existant comme modèle, puis changer son nom, ses couleurs et sa force. Dans `BOSS`, ajoute une ligne sous celle du Tyran :

```
givre: { nom: "Cheval de givre", titre: "Le présent gelé", pv: 300, degats: 18, comme: "troie", teinte: 200 },
```

Puis mets son nom au début de `ORDRE`, pour l'affronter en premier :

```
ORDRE: ["givre", "troie", "botnet", "zombie", "e404", "kernel"],
```

**Ce que tu dois voir :** en salle 7, « CHEVAL DE GIVRE » et son titre s'affichent en grand. Il a les attaques et le dessin du Cheval de Troie, en bleu glacé. Vaincu, il lâche le virus de son modèle. Pour le tester vite, baisse `pv` à 50.

**Va plus loin (codeur) :** la liste `KITS` donne les attaques de chaque boss. Retire `"ventre"` de la ligne `troie` et ton Cheval de givre n'ouvrira plus jamais son point faible (le vrai Cheval de Troie non plus, car il partage la même liste). Chaque attaque est une fonction de `ATTAQUES` : lis `martelage`, la plus courte, pour comprendre comment elles marchent.

Ouvrir BOSS →

## 10. Ça ne marche plus ?

Pas de panique : une faute de frappe arrête le jeu, mais l'atelier te dit où elle est. Le message rouge indique une ligne : la faute est sur cette ligne **ou juste au-dessus**.

| Message (en anglais) | Ce qui se passe | La solution |
|---|---|---|
| `Unexpected identifier` | Il manque une virgule à la fin de la ligne d'au-dessus. | Ajoute `,` après la valeur. |
| `Unexpected token` | Un caractère en trop ou une parenthèse, accolade ou guillemet mal fermé. | Vérifie que chaque `{` a son `}`, chaque `"` son partenaire. |
| `Invalid or unexpected token` | Un guillemet ouvert jamais fermé. | Ferme le texte : `"Bourrin 3000"`. |
| `… is not defined` | Un mot sans guillemets que le jeu ne connaît pas ( `feu` au lieu de `"feu"` ). | Ajoute les guillemets, ou corrige l'orthographe. |
| Aucune erreur, mais rien ne change | As-tu cliqué sur ▶ LANCER ? La valeur est-elle bien dans la liste des choix ? | Relance, et relis le commentaire au-dessus du réglage. |
| Écran noir | Une valeur extrême (vitesse 999 999, taille négative…). | Reviens à une valeur raisonnable, ou « Nouveau visiteur ». |

> À ÉVITER
> La virgule décimale française : `0,5` casse la ligne, écris `0.5`. Et ne supprime pas les lignes qui commencent par `function` : ce sont les recettes du jeu.

## 11. Petit glossaire

| | |
|---|---|
| **Variable / constante** | Un nom qui retient une valeur ( `let` si elle change, `const` sinon). |
| **Objet** | Une fiche `{ nom: valeur, … }`. |
| **Tableau** | Une liste `[ a, b, c ]`. |
| **Fonction** | Une recette nommée qu'on peut appeler : `mourir()`. |
| **Booléen** | `true` ou `false`. |
| **Sprite** | Une petite image d'un personnage. |
| **Hitbox** | La zone invisible qui compte pour les coups. |
| **Cooldown** | Le temps d'attente avant de pouvoir refaire une action. |
| **dt** | Le temps écoulé depuis l'image précédente. |
| **Canvas** | La zone de la page où le jeu dessine. |
| **localStorage** | La petite mémoire du navigateur, où est rangée la sauvegarde. |